

Milestone 1: Formula 1 Podium Prediction Companion

Introduction

Formula 1 generates rich, relational race-weekend data — qualifying sessions, grid penalties, pit stops, driver and constructor histories — that determines, long before lights out, which drivers have a realistic shot at the podium. Predicting podium finishes from pre-race information is a domain where careful feature engineering and leakage control (see below) matter as much as model choice, since results, points, and status are only known *after* the race.

This project implements a full data-engineering and machine-learning pipeline: converting the raw relational F1 tables into a validated, leakage-safe analytical table, and predicting whether a driver will finish on the podium using only information available after qualifying and before the race starts.

Dataset

Formula 1 World Championship (1950–2024) — Kaggle / Ergast dataset:
<https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020>

The dataset is **relational**, meaning the information you need is spread across multiple linked CSV tables rather than sitting in one file. It comprises 14 tables: `circuits`, `constructors`, `constructor_results`, `constructor_standings`, `drivers`, `driver_standings`, `races`, `results`, `qualifying`, `lap_times`, `pit_stops`, `sprint_results`, `seasons`, and `status`. You will need to join several of these tables together to answer most questions in this milestone.

- **Unit of analysis:** one driver entering one race.
- **Prediction target:** whether that driver finishes on the podium (`positionOrder <= 3`).
- **Prediction cutoff:** after qualifying, before race start. Current-race results, status, points, laps, pit stops, and post-race standings are not allowed as features – using them would be **data leakage** (using information from the future that would not actually be available at prediction time). Part of this milestone is learning to spot which fields count as “future” information yourself – see Questions 4 and 5.

Project Objectives

1. Data Cleaning & Auditing

At minimum, your cleaning must address the following (this list gives you the required checks – your own exploration of the data may turn up additional issues worth handling and documenting):

- Initial EDA, exploratory data analysis of the data.
 - Hint: this step should be done both before and after the cleaning, where you explore the impact of the cleaning on the data.
 - EDA should be supported with the proper visualization.
- Clean placeholder “missing” values, called **sentinels** (such as the literal string \N in these CSVs), and apply semantic typing to dates and durations (e.g., convert date strings to actual date objects, lap-time strings to numeric seconds).
- Your modeling table must have exactly one row for each driver in each race, because each row is one prediction: "will this driver finish on the podium in this race?"
 - In other words, the combination (raceId, driverId) must be unique. Check the data for any driver who appears more than once in the same race.
 - In early F1 (mainly the 1950s), these duplicates come from shared drives, where drivers swapped cars mid-race.
 - Choose a rule for which row to keep, apply it, and explain why you chose it.
- Verify that primary keys (the column(s) that uniquely identify each row in a table) are unique and non-null, and that foreign keys (columns that reference a primary key in another table) reconcile
 - i.e., every foreign key value actually exists in the table it points to. Raw CSV files remain immutable; cleaning happens on copies.

Note: Your EDA can not be based on manual inspection and has to be supported with implementation choices along with justification of these choices, and the visualization of them.

2. Data-Engineering Questions

To answer each question, you would need to use multi-hop joins (a conclusion from `results.csv` alone is not sufficient – you must figure out which tables to combine). The results should be the answer to each question, a visualization of the results, and a short justification of grain, missingness, and – where relevant – the pre-race cutoff.

1. Which circuits convert a front-row start into a podium most often, and does that ranking change after the 2014 hybrid era? Distinguish qualifying position from grid (which can differ due to penalties).

2. Do drivers podium more often when racing at a circuit in their home country than elsewhere, after **controlling for starting position** (i.e., comparing drivers who started from similar grid positions, so the effect isn't just because home drivers happen to qualify better)?
3. Which constructors have the highest mechanical-retirement rate in 2014–2021 vs. 2022–2024?

Note: A mechanical retirement is a race start that ended early because a component of the car failed. It excludes failures caused by a crash, by the driver, or by a regulation decision.

3. Feature Selection & Engineering

- Feature choices must be justified with exploratory analysis – report the evidence (plots, correlations, or experiments) behind each choice.
 - The justification could be empirical, meaning you tried different approaches and from which you decided on the chosen feature set, in this case you need to keep these trials in your notebook for verification
 - Or you could have a hypothesis from the earlier EDA that you decided to choose this feature set.
- Use an appropriate split (Training-Testing-Validation) for the data and justify your choice of the split.

Design room: the exact feature list and encoding choices are left to your discretion – document the reasoning for whichever you choose.

4. Predictive Modeling Task

Train and evaluate the following three models on the same modeling table (same rows, same feature set), so their scores are directly comparable:

- **At least two different statistical ML models** (e.g., logistic regression, random forest, gradient boosting).
- **A neural network with a shallow architecture** (roughly 1–2 hidden layers). “Shallow” describes the network’s depth, not how you build it – using a standard library (e.g. Keras, PyTorch) rather than writing the training loop from scratch is expected and fine.

This is a **binary classification problem**: podium vs. not (predicting the exact finishing position, e.g. 1st/2nd/3rd, is out of scope for this milestone).

The following evaluation metrics are all relevant to this task:

Metric	What it measures
ROC-AUC	Area under the receiver operating characteristic curve. Measures how

Metric	What it measures
	well the model separates the two classes overall.
PR-AUC	Area under the precision-recall curve. Summarizes how well the model ranks true podium finishers above non-finishers; more informative than ROC-AUC when the positive class (podiums) is rare.
F1 score	Harmonic mean of precision and recall at a chosen probability threshold – balances false positives against false negatives.

Compute all of the above metrics for every model.

Report performance on training/validation and on the unseen test set.

Thresholds and hyperparameters must be selected using training and validation data only – never the test set.

Design room: the exact statistical model and neural-network architecture (layer sizes, regularization) are left open, so different teams can explore different tools while still being compared on the same metrics – justify your choice against the baseline.

5. Experiments

- **Feature-group ablation:** an ablation means removing one group of features at a time (e.g., all form features, or all qualifying features) and re-training to see how much performance drops without it.
 - This step justifies each feature *before* training using exploratory evidence, while ablation measures each feature group's actual contribution *after* training. Report results for each feature group (e.g. standings, qualifying, grid).

6. Model Explainability (XAI)

- **Global explanations:** permutation importance and **SHAP**.
- **Local explanation:** at least one explanation for a single prediction (rather than the model's behavior overall), using **LIME**, local **SHAP** (e.g. a **SHAP** force/waterfall plot for one row), or both.
- All plotted and interpreted.

Important: explanations must not be presented as causal effects. A feature being important to the model is not proof that it causes podium finishes – it only means the model relies on it to make predictions. (This distinction was a common point of confusion last year, so state it explicitly in your report.)

Deliverables

1. **One** run-all Jupyter notebook containing all of the above: audit, cleaning, data-engineering answers, feature engineering, modeling, experiments, XAI, and the inference function – with every conclusion backed by a plot, statistic, or controlled experiment. (Any clear, sensible file name/location is fine – there is no required path.)
 - a. **Note:** Section your notebook to make it easier to navigate and make the evaluation run smoothly.
2. EDA exploratory data analysis along with the plots.
3. A unique driver–race modeling table with documented duplicate handling and leakage-safe form/standings/grid/qualifying features.
4. Model comparison table (statistical, shallow NN), with all five metrics from Section 4 reported on validation and on the unseen test set, and your primary/secondary metric choice justified.
5. Feature-group ablation results and at least two preprocessing-order experiment results.
6. Global (permutation importance, **SHAP**) and local (**LIME** and/or local **SHAP**) explanations, plotted and interpreted.
7. Inference function, demonstrated on a complete and a partially-missing record.
8. **Analytical report**, combining:
 - Written answers to the three data-engineering questions, with figures.
 - Selected features and why each is valid at the pre-race cutoff.
 - Known limitations (e.g., historical qualifying-coverage drift, era sensitivity).

Milestone 1 Checklist

Evaluation dates and slot-reservation details will be announced separately.

What to expect at the evaluation

1. **Every team member must be prepared.** Any question about data cleaning, feature engineering, modeling choices, evaluation metrics (including your primary/secondary choice) may be directed to any team member.
2. **Explain & justify every step in the context of your project.** Say why each step makes sense for the F1 podium-prediction task, with evidence (plots, numbers, statistics, or experiment references). “An AI tool suggested it” is not, on its own, a justification; you still need to explain and defend *why* it was the right choice for this data and task.
3. **Two legitimate ways to motivate a step:**
 - *Hypothesis-driven (EDA-supported)*: e.g. “EDA showed qualifying missingness jumps before year X, so we...” – show the EDA that led you there.
 - *Trial-and-error (empirical)*: report a clear before/after baseline for the complete pipeline, and explain why the change helped or hurt. Show each attempt in its own notebook cell/section so the before/after comparison is visible, not just described in prose.
4. **Compare multiple approaches** for anything you tried to improve (features, preprocessing, models, hyperparameters), and report comparative results. In total, your notebook should show at least three distinct modeling attempts (for example: at least two more among statistical-model variants, the neural network, and hyperparameter variants).
5. **Baseline requirement:** a **baseline** is the simplest reasonable model or approach, used as a reference point so you can tell whether a more complex method is actually earning its added complexity. The notebook must include a statistical ML model or a shallow FFNN as baseline. A different architecture requires implementing and comparing against these baselines.

Notebook & report requirements

These are requirements to satisfy, not checkbox items; the checkbox checklist itself follows in the next subsection.

- **Run-all ready:** the notebook executes top to bottom with a single Run All, no hidden manual-modification cells.
- **Correct order & documentation:** logical order, clear markdown explanations justifying choices.

- **All plots and data-engineering answers included** in both the notebook and the structured report.

Checklist

- ☐ Exploratory Data Analysis (EDA)
- ☐ Data Cleaning
 - ☐ Sentinel handling (\N), semantic typing
 - ☐ Grain fixed to one row per (raceId, driverId), duplicates documented
- ☐ Data Analysis
 - ☐ Plotting the results
- ☐ Answering the Data-Engineering Questions (all 3)
 - ☐ Plotting the results
 - ☐ Answered in markdown in the notebook, and added to the report
- ☐ Feature Selection
 - ☐ Reason for chosen features, with evidence
 - ☐ Plotting the effect of the features
 - ☐ Leakage-safe form feature implemented (using only past races)
- ☐ Data Pre-processing and Feature Engineering
 - ☐ Effect of the step on data distribution/range/values
 - ☐ Preprocessing-order experiment(s) reported
- ☐ Proper Dataset Splitting for training, validation and testing
- ☐ Prediction Model
 - ☐ Statistical model / shallow FFNN, and how it works
 - ☐ Its limitations
 - ☐ Performance during training and validation
 - ☐ Performance on unseen test data
 - ☐ Plotting the reports
 - ☐ Feature-group ablation

☐ XAI

- ☐ Global Explanation (permutation importance, **SHAP**)
- ☐ Local Explanation (**LIME**)
- ☐ Plotting and explaining in the report

Submission

Note the following about the submission:

- *The deadline for submission: 18th of October 2026 at 11:59 PM*
- *Use the Following link for submission: <https://forms.gle/esDxHsvfzffYHVAZ9>*
- *You will be submitting an **accessible** link to your Kaggle notebook*
- *You will be submitting an **accessible** GitHub repo link with your code on it*
- *You will be submitting your report in PDF format*